ASSIGNMENT-10.3

NAME:Sahastra

ROLL NO:2303A52499

BATCH:50

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:

1. Run the code and test it with factorial(5).

2. Use an AI assistant to:

o Identify the logical bug in the code.

o Explain why the bug occurs (e.g., off-by-one error).

o Provide a corrected version.

3. Compare the AI's corrected code with your own manual fix.

4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

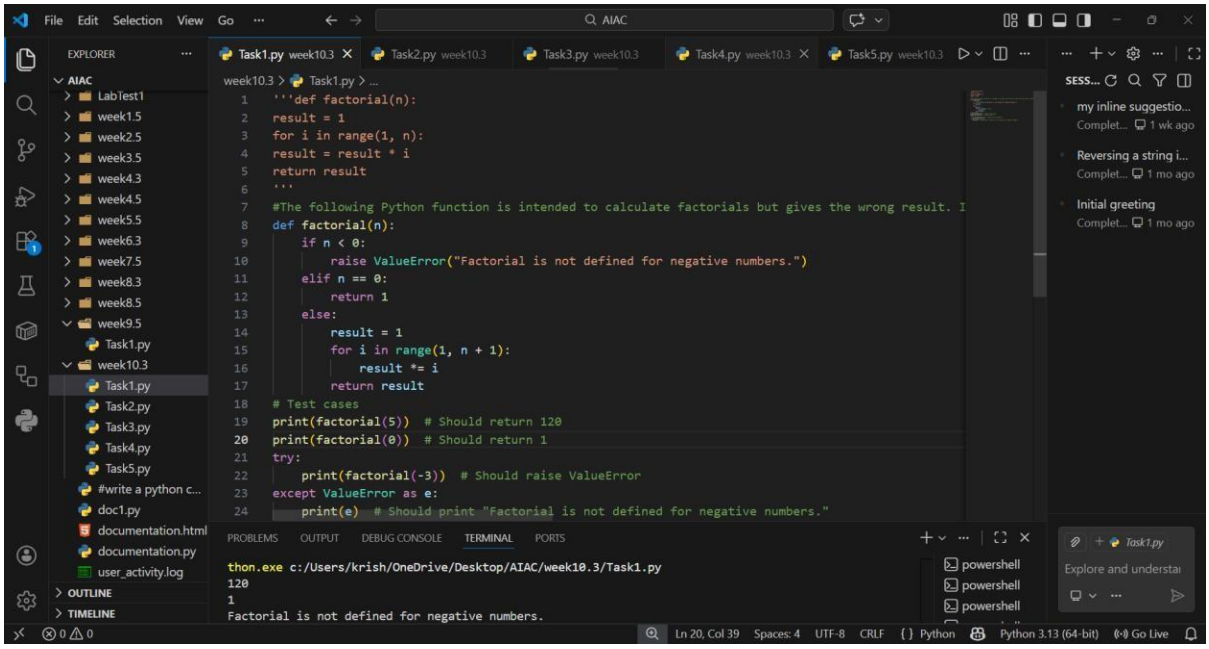
Expected Output:

Corrected function should return 120 for factorial(5).

PROMPT:

“The following Python function is intended to calculate factorials but gives the wrong result. Identify the bug, explain why it occurs, and provide a corrected version. Also, check if edge cases like zero or negative inputs are handled.”

CODE AND OUTPUT:



EXPLANATION:

This prompt explicitly asks the AI to detect the logical error (off-by-one in the loop), explain the reasoning, and fix it. It also ensures the AI considers edge cases beyond the immediate bug.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario:The following code works but is poorly written:

.

def calc(a, b, c):

if c == "add":

return a + b

elif c == "sub":

return a - b

elif c == "mul":

return a * b

elif c == "div":

Instructions:

5. Use AI to:

o Critique the function's readability, parameter naming, and

lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.

2. A complete docstring (description, parameters,

return value, examples).

3. Exception handling for division by zero.

4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by

zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles

errors gracefully.

PROMPT:

"Rewrite the following function to improve readability, naming, and documentation. Add a descriptive docstring, handle division by zero, validate inputs, and make the code more robust."

CODE:

```python
'''
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
'''
#Rewrite the above function to improve readability, naming, and documentation. Add a descriptive
def calculate(a, b, operation):
    """
    Performs a basic arithmetic operation on two numbers.
    Parameters:
    a (float): The first number.
    b (float): The second number.
    operation (str): The operation to perform. Must be one of "add", "sub", "mul", or "div".
    Returns:
    float: The result of the arithmetic operation.
    Raises:
    ValueError: If the operation is not recognized or if division by zero is attempted.
    TypeError: If the inputs are not numbers.
    """
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise TypeError("Both a and b must be numbers.")
    if operation == "add":
        return a + b
    elif operation == "sub":
        return a - b
```

```python
def calculate(a, b, operation):
    elif operation == "mul":
        return a * b
    elif operation == "div":
        if b == 0:
            raise ValueError("Cannot divide by zero.")
        return a / b
    else:
        raise ValueError("Invalid operation. Supported operations are: add, sub, mul, div.")
# Test cases
print(calculate(10, 5, "add"))  # Should return 15
print(calculate(10, 5, "sub"))  # Should return 5
print(calculate(10, 5, "mul"))  # Should return 50
print(calculate(10, 5, "div"))  # Should return 2.0
try:
    print(calculate(10, 0, "div"))  # Should raise ValueError
except ValueError as e:
    print(e)  # Should print "Cannot divide by zero."
try:
    print(calculate(10, 5, "mod"))  # Should raise ValueError
except ValueError as e:
    print(e)  # Should print "Invalid operation. Supported operations are: add, sub, mul, div."
try:
    print(calculate("10", 5, "add"))  # Should raise TypeError
except TypeError as e:
    print(e)  # Should print "Both a and b must be numbers."
```

OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python313\py
thon.exe c:/Users/krish/OneDrive/Desktop/AIAC/week10.3/Task2.py
15
5
50
2.0
Cannot divide by zero.
Invalid operation. Supported operations are: add, sub, mul, div.
Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>


Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>

Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>
Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

This prompt guides the AI to critique the original code and produce a professional, well-documented version with error handling. It ensures the AI doesn't just fix syntax but also improves clarity and usability.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.

9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:

o List all PEP8 violations.

o Refactor the code (function name, spacing, indentation, naming).

10. Apply the AI-suggested changes and verify functionality is preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
for i in range(2, n):
```

if n % i == 0:

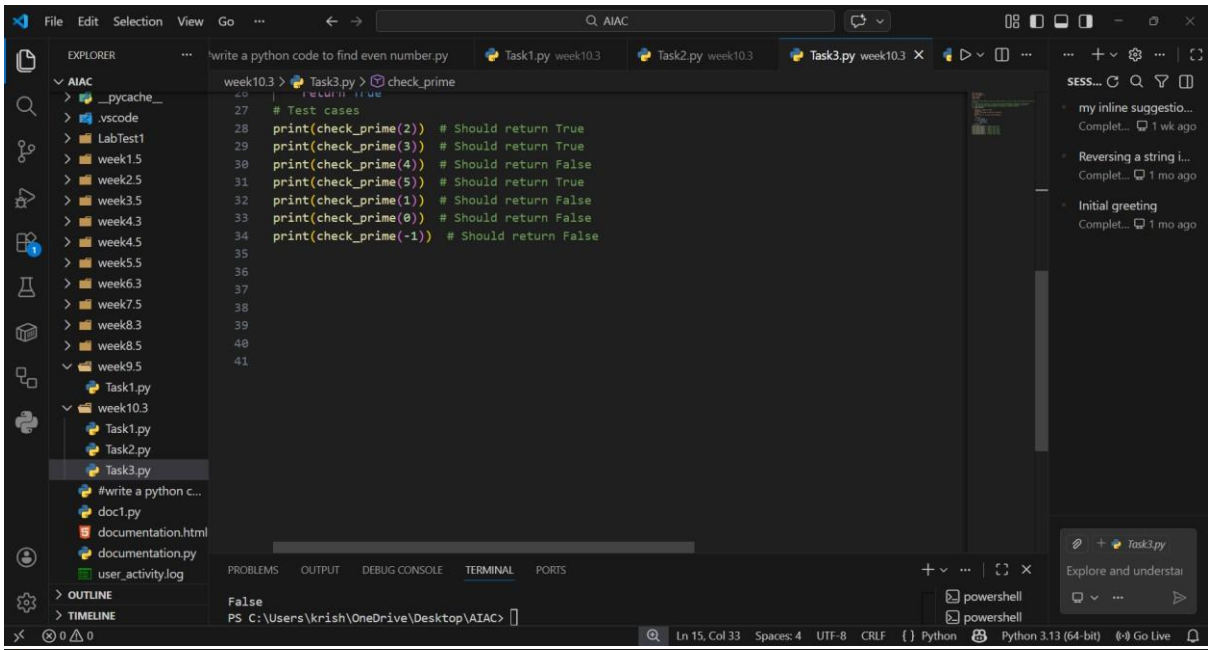
return False

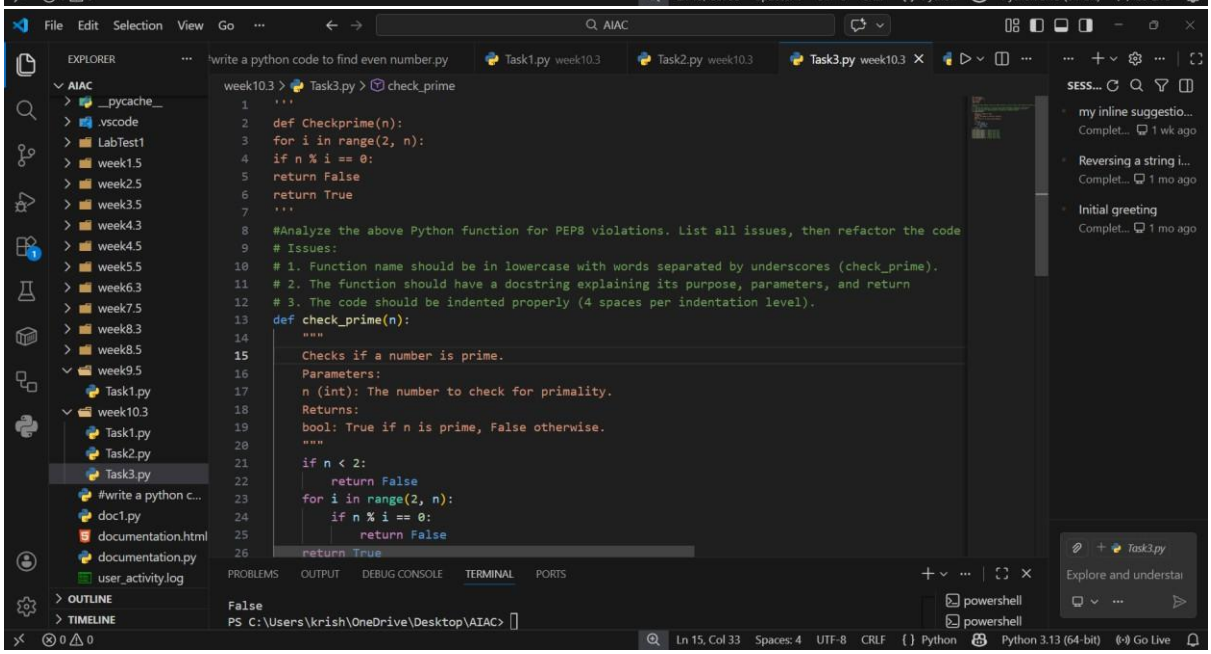
return True

PROMPT:

“Analyze the above Python function for PEP8 violations. List all issues, then refactor the code to be fully PEP8-compliant while preserving functionality."

CODE:





OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python313\py
thon.exe c:/Users/krish/OneDrive/Desktop/AIAC/week10.3/Task3.py
True
True
False
True
False
False
False
```

EXPLANATION:

This prompt ensures the AI acts like a linter, pointing out violations (naming, spacing, indentation) and then refactoring. It keeps the focus on compliance while maintaining correctness.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

def processData(d):

return [x * 2 for x in d if x % 2 == 0]

Instructions:

1. Manually review the function for:

o Readability and naming.

o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

from typing import List, Union

def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:

if not isinstance(numbers, list):

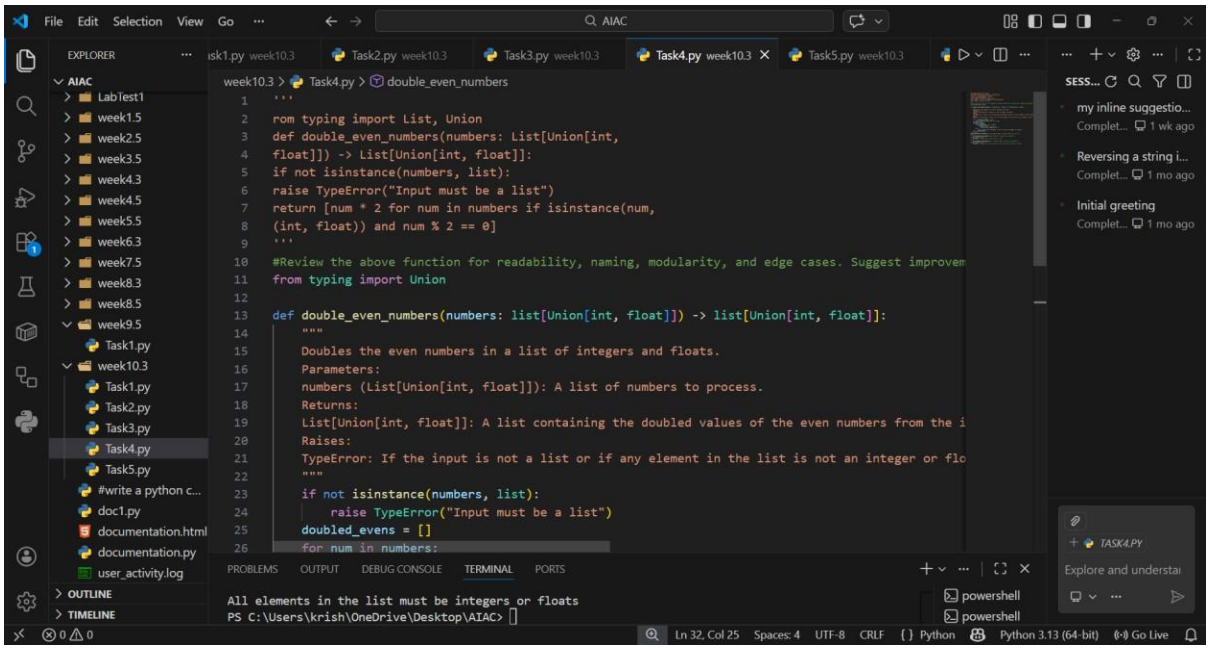
raise TypeError("Input must be a list")

return [num * 2 for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]

PROMPT:

"Review the above function for readability, naming, modularity, and edge cases. Suggest improvements with type hints, input validation, and generalization. Then refactor the function accordingly."

CODE:



```python
def double_even_numbers(numbers: list[Union[int, float]]) -> list[Union[int, float]]:
    for num in numbers:
        if isinstance(num, (int, float)):
            if num % 2 == 0:
                doubled_evens.append(num * 2)
        else:
            raise TypeError("All elements in the list must be integers or floats")
    return doubled_evens
# Test cases
print(double_even_numbers([1, 2, 3, 4.0, 5, 6]))  # Should return [4, 8.0, 12]
try:
    print(double_even_numbers("not a list"))  # Should raise TypeError
except TypeError as e:
    print(e)  # Should print "Input must be a list"
try:
    print(double_even_numbers([1, 2, "three", 4]))  # Should raise TypeError
except TypeError as e:
    print(e)  # Should print "All elements in the list must be integers or floats"
```

OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python313\python.exe c:/Users/krish/OneDrive/Desktop/AIAC/week10.3/Task4.py
[4, 8.0, 12]
Input must be a list
Input must be a list
All elements in the list must be integers or floats
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

This prompt positions the AI as a reviewer, not just a fixer. It ensures the AI provides critique, suggests enhancements, and produces a refactored version with clearer intent and robustness.

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).

2. Use AI to:

o Analyze time complexity.

o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).

o Provide an optimized version.

3. Compare execution time before and after optimization.

4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

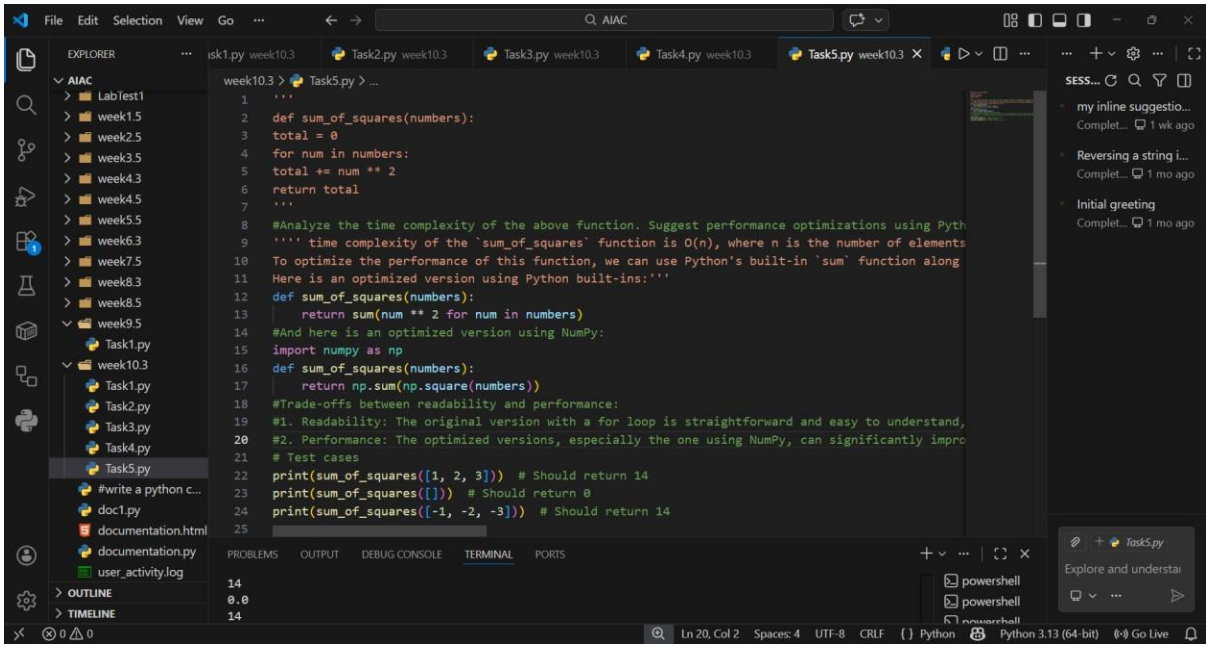
def sum_of_squares_optimized(numbers):

return sum(x * x for x in numbers)

PROMPT:

"Analyze the time complexity of the above function. Suggest performance optimizations using Python built-ins or libraries like NumPy. Provide an optimized version and discuss trade-offs between readability and performance."

CODE AND OUTPUT:



EXPLANATION:

This prompt ensures the AI evaluates efficiency, proposes alternatives, and explains trade-offs. It balances optimization with maintainability, which is crucial in real-world projects.